Code of Conduct:

I AKSHAYA SEKAR (1911260204) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: AKSHAYA SEKAR

SIMATS ENGINEERING

CO2_AT1 SCENARIO-BASED ASSESSMENT

COURSE NAME: Software Testing

COURSE CODE: CSA3711

COURSE OUTCOME COVERED: CO2 – Design effective test cases using black-box and white-box testing techniques and control method flow.

ASSESSMENT TOOL WEIGHTAGE: 50%

QUESTIONS AND ANSWERS

QUESTION 1: An online banking system is experiencing frequent failures during fund transfers, especially at boundary values. Analyze and redesign the test strategy.

The repeated failures near minimum and maximum transfer limits suggest that the existing tests do not adequately cover boundary conditions, or the rules applied at those limits. Random testing may find unexpected problems, but it can miss important values unless the sample happens to include them. The transfer requirement should first define the permitted minimum and maximum amounts, currency precision, account balance rules, daily limits, and any fees.

Equivalence Class Partitioning (ECP): Divide transfer amounts into valid values within the permitted range and invalid values below the minimum or above the maximum. Include separate classes for zero, negative amounts, non-numeric input, excessive decimal places, insufficient balance, and a transfer that exceeds a daily limit where those inputs are applicable.

Boundary Value Analysis (BVA): If the permitted range is [L, U], test L−1 smallest unit, L, L+1 smallest unit, a representative middle value, U−1 smallest unit, U, and U+1 smallest unit. For currency, the smallest unit is normally the smallest supported currency increment. Also test balances exactly equal to the transfer amount and just below it.

Random testing should be retained as supplementary exploratory testing, not used as the main strategy. A structured suite should prioritize ECP, BVA, decision-table cases for interacting rules, and end-to-end checks that confirm the account balance and transaction history are updated consistently. Each test should state its preconditions, input, expected result, and postcondition. This approach is more repeatable and gives stronger coverage of known risk areas than random samples alone.

QUESTION 2: A healthcare application has critical post-deployment defects due to insufficient coverage. Explain how to use a conceptual control flow graph and improve coverage.

The production defects indicate that the earlier test suite may have executed some statements without exercising all decision outcomes or important execution paths. A control flow graph (CFG) represents the program as nodes for statements or blocks and directed edges for possible transfers of control.

A conceptual healthcare workflow could be represented as: Start → validate patient/request → decision “input valid?”; the invalid branch returns a validation error, while the valid branch continues to a decision “authorization permitted?”; denial returns an access error, while approval performs the operation → records the result → End. Additional nodes and branches should be added for missing records, exceptional

responses, and recovery handling if these exist in the actual code. This is a conceptual model only; the precise CFG must be derived from the module source code.

Statement coverage confirms that statements execute at least once, but it does not ensure that both outcomes of each decision are exercised. Branch coverage tests each decision outcome. Path coverage attempts to exercise distinct sequences of branches, although loops and complex logic can make complete path coverage impractical or infinite.

The improved strategy is to derive tests from the actual CFG, test both outcomes of every decision, cover critical independent paths, and include invalid input, denied authorization, missing data, exceptions, and recovery paths. Measure statement and branch coverage, review uncovered code, and prioritize risk-critical paths. Pair white-box tests with requirement-based black-box tests and regression tests. This makes the coverage claim measurable and reduces the chance that important decision outcomes remain untested.

QUESTION 3: A retail e-commerce system uses price, quantity, and discount inputs, but errors occur in combinations. Design a stronger black-box test strategy.

Testing each field separately is insufficient because the final amount depends on interactions among price, quantity, and discount. A test suite should combine equivalence classes, boundary values, and decision-table testing.

For price, define valid positive prices and invalid negative, zero-if-prohibited, missing, non-numeric, or over-precision values according to the requirements. For quantity,

define valid positive integers and invalid zero, negative, fractional, missing, and excessively large values. For discount, define allowed percentage or amount ranges, including the lower and upper limits and values just outside them. Exact classes depend on the product rules.

BVA should test values immediately below, at, and above each defined limit. Decision tables should then combine meaningful conditions, such as price valid/invalid, quantity valid/invalid, discount within range/out of range, and discount exceeding the subtotal. Expected actions can include calculating the total, rejecting the input, or displaying a validation message. Include cases where the discount is exactly equal to the subtotal and just above it, and confirm that the total never becomes negative if the business rule prohibits that result.

Testing every possible combination can be unnecessarily large. Use a decision table to cover all distinct business rules, then pairwise or risk-based combination testing for remaining interactions where appropriate. Validate both the displayed total and the amount sent to checkout or payment processing. The resulting suite is more complete than isolated field tests while keeping the number of cases manageable.

QUESTION 4: A module has nested conditions and loops but only partial path coverage. Explain the gaps and propose a path-testing strategy.

Partial path coverage may result from an incomplete CFG, missed decision outcomes, or failure to identify independent paths through nested conditions and loops. If tests execute each statement but do not exercise different combinations of decisions, defects can remain hidden in control-flow interactions.

Construct a CFG from the actual source code, with nodes for processing blocks and decisions, and edges for each possible outcome. Identify the entry and exit nodes, loop headers, loop exits, and exception edges where relevant. Cyclomatic complexity can estimate the number of linearly independent paths using $M = E - N + 2P$, where E is the number of edges, N is the number of nodes, and P is the number of connected components (normally 1 for a single routine). The value guides a basis set of independent paths; it does not mean every possible path in a loop has to be enumerated.

Design tests to cover each independent path and each branch outcome. For loops, test zero iterations if allowed, one iteration, multiple iterations, and boundary/termination conditions. For nested loops, use representative combinations and risk-based tests rather than attempting every unbounded combination. Include tests for early returns, exceptions, and invalid data. Track statement, branch, and basis-path coverage, review uncovered edges, and rerun tests after changes. This provides a defensible coverage strategy and improves the chance of detecting errors in control flow and loop termination.

QUESTION 5: A financial application uses both black-box and white-box testing, yet production defects continue. Propose an improved hybrid strategy.

The continued defects suggest that the two approaches may not be connected to the same requirements and risks, or that coverage of metrics are being treated as proof of correctness. Black-box testing checks externally visible behavior against requirements; white-box testing examines internal decisions, branches, and paths. Neither is sufficient alone.

Start with a risk-based requirement-to-test traceability matrix for financial calculations, transaction integrity, authorization, rounding, and error handling. Use ECP to identify valid and invalid input classes and BVA for amount, balance, rate, date, and limit boundaries. Use decision tables for combinations of business rules such as fees, discounts, account status, and transaction limits. Define expected outputs independently from the implementation.

From the source code, construct CFGs for high-risk routines and test every decision outcome, critical independent path, and relevant exception path. Review statement and branch coverage and use basis-path analysis where appropriate. Add integration tests for database updates, duplicate requests, timeouts, rollback, and retry behavior, plus regression tests for each production defect.

Coverage percentages should be interpreted alongside requirement coverage, defect trends, and risk. A high statement-coverage figure does not prove that all business rules or input combinations are correct. The hybrid approach improves defect detection by connecting external requirements to internal control flow and validating both the calculation and its side effects.

QUESTION 6: An HR payroll system calculates bonuses by years of service and performance rating; errors occur exactly at 5, 10, and 15 years. Redesign the test suite using BVA and ECP.

The failures at exactly 5, 10, and 15 years strongly suggest that bonus tiers or comparison operators may be implemented incorrectly at transition points. The requirements must specify whether each threshold belongs to the lower or upper service band and how partial years are treated.

Use ECP to partition service into ranges separated by 5-, 10-, and 15-year thresholds, plus invalid classes such as negative service, missing values, non-numeric values, and unsupported precision. Partition performance ratings into each valid rating category and invalid values. Test representative values within every class.

For each threshold T, apply BVA using $T-\varepsilon$, T, and $T+\varepsilon$, where ε is the smallest meaningful service increment supported by the system. If service is stored in whole years, test 4, 5, and 6; similarly test 9, 10, 11 and 14, 15, 16. If the system uses months or days, use the appropriate smallest unit instead. Combine each threshold test with representative low, middle, and high-performance ratings, especially where bonus rules differ by rating.

Exploratory testing can uncover unexpected behavior, but it may not systematically check every exact threshold. BVA and ECP are more reproducible for this rule-based calculation. Compare payouts with an independently calculated expected result, verify rounding and payroll totals, and retain every discovered defect as a regression test.

QUESTION 7: A laboratory temperature controller fails to trigger an alarm during a sensor-malfunction sequence. Explain CFG, branch coverage, and MC/DC testing.

Because this is a safety-related controller, tests must verify not only normal heating and cooling but also sensor faults and alarm behavior. A conceptual CFG can begin with reading the sensor, then decide whether the reading is valid. An invalid reading branches to a safe-state action and alarm decision; a valid reading branches to comparisons against low and high temperature limits, leading to heater-on, cooler-on,

or neither action. The flow then updates status and repeats. This is illustrative; the actual CFG must follow the real controller logic.

Branch coverage requires each decision of outcome to be exercised, including sensor-valid/invalid and relevant temperature comparison. However, branch coverage may not prove that each individual condition independently affects a compound decision.

For 100% Modified Condition/Decision Coverage (MC/DC), identify every Boolean condition in each compound decision. Construct pairs of tests where one condition changes while the other conditions are held fixed, demonstrating that the changed condition independently changes the decision outcome. For example, if alarm activation depends on sensor fault, temperature danger, and controller enabled status, tests must demonstrate each condition's independent effect according to the specified safety logic. Do not assume a particular truth table without the actual requirements.

Also test sensor disconnection, stale readings, out-of-range readings, recovery after a fault, and safe behavior when readings are unavailable. Use requirements-based expected results, simulated sensor inputs, code coverage reports, and independent review of safety-critical paths. MC/DC strengthens evidence that compound decisions have been tested systematically, but it must be combined with hazard analysis and system-level safety validation.

QUESTION 8: A travel portal allows multiple promo codes and sometimes produces a negative total. Apply decision-table testing or cause-effect graphing.

Testing each promo code independently misses interactions such as incompatible offers, stacking limits, discounts exceeding the subtotal, and user-tier restrictions. First clarify the rules: whether codes can be combined, the order in which discounts apply, whether discounts are percentages or fixed amounts, and whether a minimum payable amount is enforced.

Create decision-table conditions for base price valid, promo code valid, code combination permitted, user tier eligible, minimum purchase met, and calculated discount less than or equal to the permitted subtotal. Actions should include applying the discount, rejecting a code for a clear reason, or capping/rejecting a result according to the documented rule. Include representative cases for no code, one valid code, two compatible codes, incompatible codes, expired or invalid codes, ineligible user tiers, and a discount equal to or greater than the subtotal.

For example, a valid price with an eligible code and a permitted combination should follow the documented discount rule; an incompatible combination should be rejected or handled according to the requirements. The exact expected amount must come from the portal's business rules, not an assumption. Verify that the total cannot become negative if that is prohibited and check displayed totals against checkout and payment requests.

Remove redundant rows only when they exercise the same rule and outcome. A compact decision table plus boundary tests for price and discount limits gives stronger interaction coverage than testing each input in isolation.

QUESTION 9: A legacy database migration tool has high statement coverage but infinite-loop crashes. Propose path testing using independent paths, loop boundaries, and cyclomatic complexity.

Statement coverage can be high even if tests do not exercise loop exit conditions, combinations of nested-loop decisions, empty datasets, malformed records, or failure paths. An infinite-loop defect often involves a termination condition or loop variable that does not progress as expected.

Build a CFG for the record-validation routine from the source code, showing loop entry, record checks, nested-loop transitions, error handling, and exits. Calculate cyclomatic complexity with $M = E - N + 2P$. This estimates the number of independent paths needed for a basis-path suite, but it does not enumerate all possible loop iterations.

Test zero records, one record, multiple records, and dataset sizes around any configured batch or buffer boundary. For each loop, test zero iterations if permitted, one iteration, several iterations, and termination at the final record. For nested loops, include cases that exercise each exit and continuation condition. Include invalid records, duplicate keys, database errors, and interruption/restart behavior. Use timeouts and iteration limits in tests to detect non-termination safely.

Measure branch and basis-path coverage and inspect whether every loop has a reachable exit under valid and invalid inputs. Add regression tests for every infinite-loop defect and verify that migration counts and data integrity remain correct. This approach addresses control flow and termination behavior that statement coverage alone cannot establish.

QUESTION 10: A food delivery tracker gets stuck when a driver cancels mid-delivery. Redesign testing with State Transition Testing.

Equivalence classes and boundary values are useful for input validation, but they do not fully test a workflow whose behavior depends on the current state and event sequence. The tracker should be modeled as a state machine with states such as Ordered, Preparing, Out for Delivery, Delivered, and a suitable cancellation or exception state.

List valid transitions and events. Examples include Ordered → Preparing, Preparing → Out for Delivery, and Out for Delivery → Delivered. Define what should happen if cancellation occurs before preparation, during preparation, or while the order is out for delivery. The requirements must determine whether the order becomes Cancelled, returns to reassignment, or follows a refund/recovery workflow. Delivered should not silently transition to an ordinary cancellation state unless the business rules explicitly allow it.

Test each valid transition, every important event in each relevant state, and invalid transitions such as a second delivery-completion event or cancellation after final completion. Test sequences where the driver cancels, network connectivity drops, a replacement driver is assigned, or the app is reopened after a state change. Verify that server state, customer display, driver status, and notifications remain consistent.

State transition coverage can be measured by covering all defined transitions; transition-pair coverage can check consecutive transition combinations. The expected state and recovery behavior must be specified for cancellation so that the tracker cannot remain indefinitely in an undefined state.

QUESTION 11: A cybersecurity authentication module has brute-force vulnerabilities despite testing. Propose a hybrid strategy combining error guessing and path analysis.

The detected brute-force vulnerability suggests that tests may have focused on successful login and ordinary invalid credentials without sufficiently examining repeated failures, timing, rate limits, and interactions among authentication conditions. Black-box security testing should evaluate observable behavior, while white-box path analysis can inspect the internal logic that applies security controls.

Use error guessing to derive cases such as repeated incorrect passwords, repeated or expired OTPs, OTP reuse, requests arriving rapidly, account-lockout boundary conditions, inconsistent IP-location signals, and recovery after a lockout. Test password, OTP, and location conditions individually and in combinations. Confirm that error messages do not reveal unnecessary account information and that throttling and logging behave as required.

Construct a CFG for the authentication flow from the actual code: input validation, password verification, OTP validation, risk/location checks, failure counting, rate limiting, lockout, and successful session creation. Identify independent paths through success, each failure type, lockout, and recovery. Check that every path that should enforce rate limiting or lockout actually does so and that there is no alternate route that bypasses a control.

Use authorized test accounts and a controlled environment. Combine requirement-based tests, branch/path coverage, security-focused negative tests, and regression checks. Do not infer that a module is secure merely because code coverage is high;

validate the observable security requirements and review logs and session behavior as well.

QUESTION 12: An e-learning platform has upgrade errors on the subscription expiration date. Apply ECP to date ranges and subscription types.

The defect occurs at the boundary between an active subscription and an expired subscription. The current suite may not distinguish dates immediately before, on, and after expiration, or may not cover how an upgrade interacts with the current tier and course access rules.

Partition dates into clearly defined classes: before the expiration date, exactly on the expiration date, and after expiration. The exact meaning of "on the expiration date" must be defined, including time zone and whether expiration occurs at the start or end of that day. Partition subscription types into Free, Basic, and Premium, and include supported and unsupported course types or upgrade paths.

Test each date class with each relevant subscription type and upgrade action. Include an upgrade initiated just before expiration that completes after the boundary, an upgrade on the expiration date, a renewal after expiration, and a user who has no active paid subscription. Verify access rights, billing, renewal date, course availability, and user-facing status. Test time-zone and daylight-saving boundaries if applicable to the system.

ECP reduces duplication by selecting representative values from each class, but the critical date boundary should also receive BVA tests immediately before, at, and after the exact expiration timestamp. Use a decision table to capture tier, course eligibility,

and date combinations. These tests make expected access and billing behavior explicit and help prevent gaps during subscription transitions.

QUESTION 13: A smart-home lighting controller uses switch cases and complex Boolean expressions. Explain why statement coverage is insufficient and design branch/condition tests.

Statement coverage only demonstrates that each statement executed at least once. A switch case or compound Boolean expression can contain untested outcomes even when all statements appear covered. This may leave a sensor range, default case, or combination of conditions untested.

Construct a CFG from the controller code, representing sensor reading, switch-case alternatives, Boolean decisions, dimming calculations, and hardware output commands. For each switch, test every supported case and the default or invalid-input path. For each decision, test both true and false outcomes. Condition coverage should make each individual Boolean condition evaluate to both true and false. Where conditions interact, consider MC/DC to show each condition's independent effect on the overall decision.

Use representative ambient-light readings across each defined range, values at and around thresholds, missing or implausible sensor readings, and transitions between lighting modes. Verify both the calculated dimming level and the actual command sent to the lighting hardware. Include sensor failure and communication failure scenarios if supported by the design.

Combine statement, branch, and condition coverage with black-box tests derived from lighting requirements and integration tests using the real or simulated hardware interface. Coverage reports should be reviewed for uncovered switch alternatives and decision outcomes. This reduces the chance that a logical path or hardware command is untested, though it does not by itself prove full system safety.

QUESTION 14: A logistics system fails to detect zero and negative shipping weights. Apply robust Boundary Value Analysis.

The missed zero and negative values indicate inadequate invalid-boundary testing. The system should define the minimum acceptable shipping weight, the supported precision, and whether zero weight is ever valid for a special shipment type. Testing should follow those requirements rather than assume all cases share the same rule.

If the minimum positive supported weight is L and the maximum is U, test values immediately below L, at L, and immediately above L; similarly test immediately below U, at U, and immediately above U. Robust BVA should also include invalid values such as zero, a negative weight, a missing value, non-numeric text, and excessive decimal precision. If L is greater than zero, include $-\varepsilon$, 0, $L-\varepsilon$, L, and $L+\varepsilon$ using the smallest supported unit ε. Test very large values and values that could overflow calculations.

Combine weight cases with representative destination zones and delivery speeds, especially where shipping rules or surcharges differ. For invalid input, verify that the system rejects the request with a clear message and does not create a shipment or charge. For valid input, independently check the computed shipping cost.

Negative scenarios are essential because real users, integrations, or malformed requests can supply unexpected data. Validation should occur at the appropriate system boundary, not only in the user interface. Add regression tests for zero and negative values and verify that invalid inputs cannot reach downstream billing or logistics processes.

QUESTION 15: Medical imaging results are inconsistent for prime-sized datasets. Propose white-box testing with Basis Path Testing.

Random testing is not enough for a defect that appears only for prime-number dataset sizes. The pattern suggests a possible assumption in array partitioning, parallel chunking, loop bounds, remainder handling, or synchronization. This is a hypothesis to investigate, not a confirmed cause without source code and execution evidence.

Inspect the algorithm and construct CFGs for data partitioning, worker allocation, loop control, and result aggregation. Use cyclomatic complexity to estimate a set of independent basis paths, then design tests that execute each path. Include dataset sizes of 0 or 1 where permitted, small composite sizes, prime sizes, sizes divisible by the worker count, and sizes that leave a remainder. Include sizes around chunk and buffer boundaries and compare serial and parallel outputs.

For each dataset, compare results with a trusted reference implementation or independently computed expected values. Verify that every element is processed exactly as intended, that no element is omitted or duplicated, and that the final aggregation is deterministic. Exercise scheduling variations and repeat runs to investigate concurrency-related inconsistencies.

Basis Path Testing provides structured control-flow coverage, but mathematical accuracy also requires suitable test oracles, numerical tolerance rules, and domain-specific validation. Retain randomized and stress testing as supplementary techniques, particularly for concurrency, while using carefully selected structured cases to target the prime-size pattern. Record the failing inputs and make them regression tests.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification